

UT 02.3 – React con TypeScript

En este tema introduciremos TypeScript y lo utilizaremos para crear un componente de React fuertemente tipado. Comenzaremos por comprender qué aporta TypeScript frente a JavaScript, aprenderemos sus tipos básicos y a crear nuestros propios tipos, veremos cómo funciona su compilador y, por último, aplicaremos todo ello al componente **Alert** creado en los temas UT 02.1 y UT 02.2.

En este tema utilizaremos las siguientes herramientas:

- **TypeScript Playground**: sitio web (<https://www.typescriptlang.org/play>) que permite experimentar con las características de TypeScript sin necesidad de instalarlo.
- **Visual Studio Code**, **Node.js** y **npm**: los mismos que en los temas anteriores.

Comprender las ventajas de TypeScript

En esta sección, comenzaremos por entender qué es TypeScript, cómo se relaciona con JavaScript y cómo permite a los equipos ser más productivos.

Comprender TypeScript

TypeScript se lanzó por primera vez en 2012 y sigue desarrollándose activamente, con nuevas versiones cada pocos meses. Pero ¿qué es TypeScript y cuáles son sus ventajas?

A menudo se hace referencia a TypeScript como un superconjunto o una extensión de JavaScript, ya que cualquier característica de JavaScript está disponible en TypeScript. A diferencia de JavaScript, TypeScript no se puede ejecutar directamente en un navegador, sino que primero debe transpilarse a JavaScript.

TypeScript añade un completo sistema de tipos a JavaScript. Se utiliza a menudo con marcos de trabajo *frontend* como Angular, Vue y React. TypeScript también se puede utilizar para desarrollar un *backend* con Node.js, o con entornos de ejecución de servidor más recientes como Bun o Deno. Esto demuestra lo flexible que es su sistema de tipos.

Cuando una base de código JavaScript crece, puede resultar difícil de leer y mantener. El sistema de tipos de TypeScript resuelve este problema: permite que los editores de código detecten errores de tipos a medida que los desarrolladores escriben código problemático, y ofrece funciones de productividad como una navegación robusta por el código y la refactorización.

Novedad (2026): TypeScript 6 y TypeScript 7

En marzo de 2026 se publicó **TypeScript 6.0**, una versión «puente» que moderniza los valores por defecto del compilador (por ejemplo, el modo `strict` pasa a estar activado por defecto) y declara obsoletas varias opciones antiguas.

En julio de 2026 se publicó **TypeScript 7.0**, cuyo compilador ha sido reescrito en el lenguaje Go. El lenguaje es el mismo, pero las compilaciones completas son habitualmente entre 8 y 12 veces más rápidas. Las opciones declaradas obsoletas en la versión 6 producen ya errores en la versión 7.

A continuación, veremos un ejemplo de cómo TypeScript detecta un tipo de error que JavaScript no puede detectar.

Detección temprana de errores de tipo

La información de tipos ayuda al compilador de TypeScript a detectar errores de tipos. En editores de código como Visual Studio Code, un error de tipo se subraya en rojo inmediatamente después de que el desarrollador lo haya cometido. Sigue estos pasos para ver un ejemplo:

1. Abre Visual Studio Code en la carpeta que prefieras.
2. Crea un nuevo archivo llamado **calculateTotalPrice.js** seleccionando la opción **Nuevo archivo** en el panel **Explorador**.

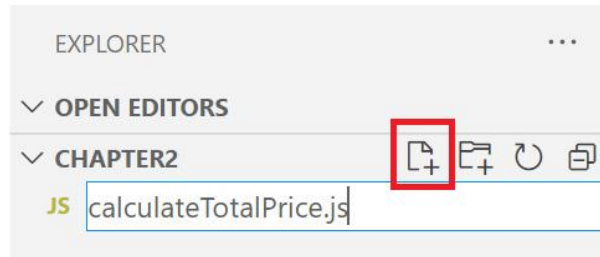


Figura 2.3.1 – Creación de un nuevo archivo en Visual Studio Code

3. Introduce el siguiente código en el archivo:

```
function calculateTotalPriceJS(
  product,
  quantity,
  discount,
) {
  const priceWithoutDiscount =
    product.price * quantity;
  const discountAmount =
    priceWithoutDiscount * discount;
  return (
    priceWithoutDiscount -
    discountAmount
  );
}
```

La función calcula el precio total de un producto a partir del producto, la cantidad y el descuento que se le pasan como parámetros. Hay un error en el código que puede ser difícil de detectar, y Visual Studio Code no lo resaltará. El error quedará claro en el paso 5, una vez que el código se haya convertido a TypeScript.

4. Ahora, crea una copia del archivo, pero con la extensión **.ts** en lugar de **.js**.

Nota

La extensión de archivo **.ts** indica que se trata de un archivo TypeScript. Esto significa que el compilador de TypeScript realizará la comprobación de tipos en este archivo.

5. En el archivo **calculateTotalPrice.ts**, elimina «JS» al final del nombre de la función y añade las anotaciones de tipo que se indican a continuación:

```
function calculateTotalPrice(
  product: { name: string; unitPrice: number },
  quantity: number,
  discount: number,
) {
  const priceWithoutDiscount = product.price * quantity;
  const discountAmount = priceWithoutDiscount * discount;
  return priceWithoutDiscount - discountAmount;
}
```

Aquí hemos añadido **anotaciones de tipo** de TypeScript a los parámetros de la función. Lo importante es que ahora el error de tipo aparece resaltado con un subrayado ondulado rojo:

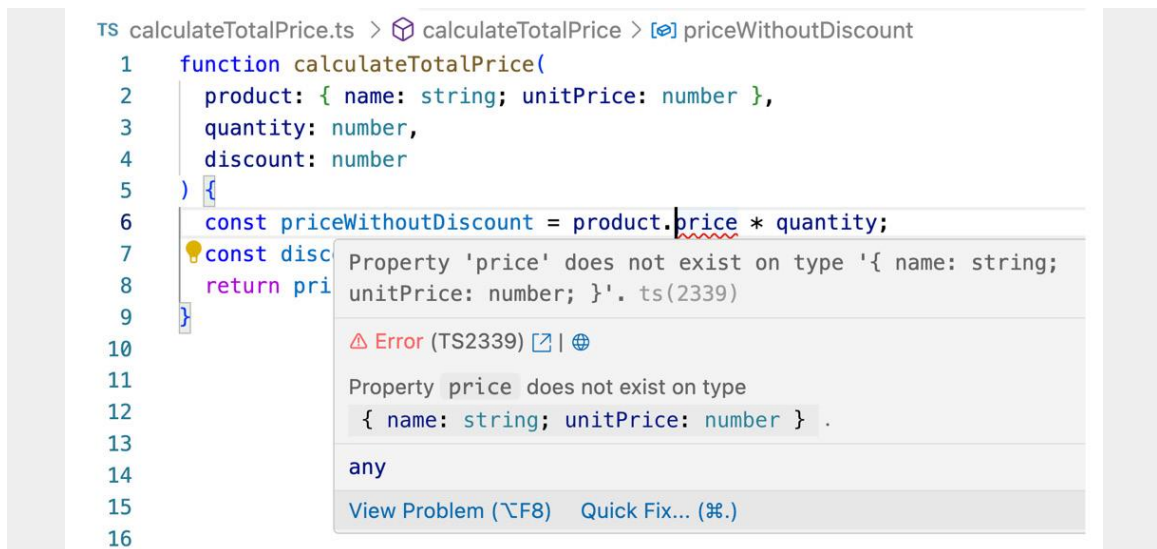


Figura 2.3.2 – Error de tipo resaltado

El error es que la función hace referencia a una propiedad **price** en el objeto **product** que no existe. La propiedad a la que se debería hacer referencia es **unitPrice**.

Detectar estos problemas en una fase temprana del desarrollo aumenta el rendimiento del equipo y supone una tarea menos para el control de calidad. Podría haber sido peor: el error podría haberse colado en la aplicación en producción y haber proporcionado a los usuarios una mala experiencia.

Mantén estos archivos abiertos en Visual Studio Code, ya que a continuación veremos cómo TypeScript mejora la experiencia del desarrollador.

Mejorar la experiencia y la productividad del desarrollador con IntelliSense

IntelliSense es una función de los editores de código que proporciona información útil sobre los elementos del código y permite completarlo rápidamente. Por ejemplo, IntelliSense puede mostrar la lista de propiedades disponibles en un objeto.

Sigue estos pasos para comprobar que TypeScript funciona mejor con IntelliSense que JavaScript. Como parte de este ejercicio, corregiremos el error del precio de la sección anterior:

1. Abre **calculateTotalPrice.js** y, en la línea 7, donde se hace referencia a **product.price**, elimina **price**. A continuación, con el cursor después del punto (.), pulsa **Ctrl + barra espaciadora**. Esto abre IntelliSense de Visual Studio Code:

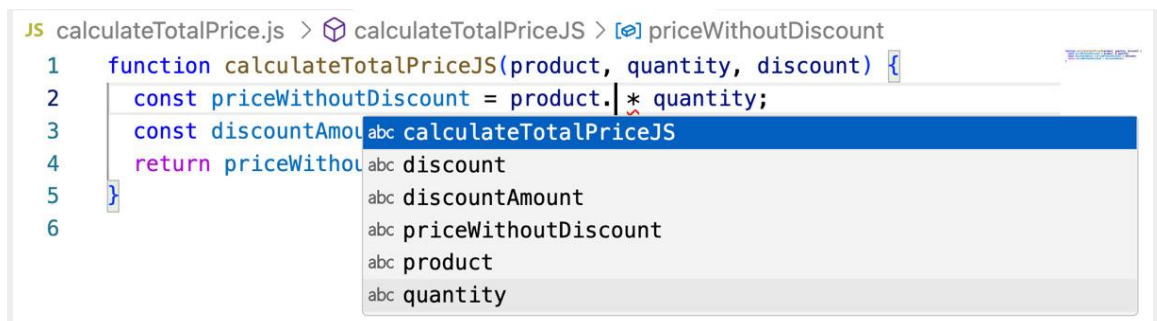


Figura 2.3.3 – IntelliSense en un archivo JavaScript

Visual Studio Code solo puede adivinar el nombre de la propiedad, por lo que muestra una lista de los nombres de variables y funciones que ha encontrado en el archivo. Por desgracia, IntelliSense no ayuda en este caso porque el nombre correcto de la propiedad, **unitPrice**, no aparece en la lista.

- Ahora, abre **calculateTotalPrice.ts**, elimina **price** de **product.price** y pulsa **Ctrl + barra espaciadora** para abrir IntelliSense de nuevo:

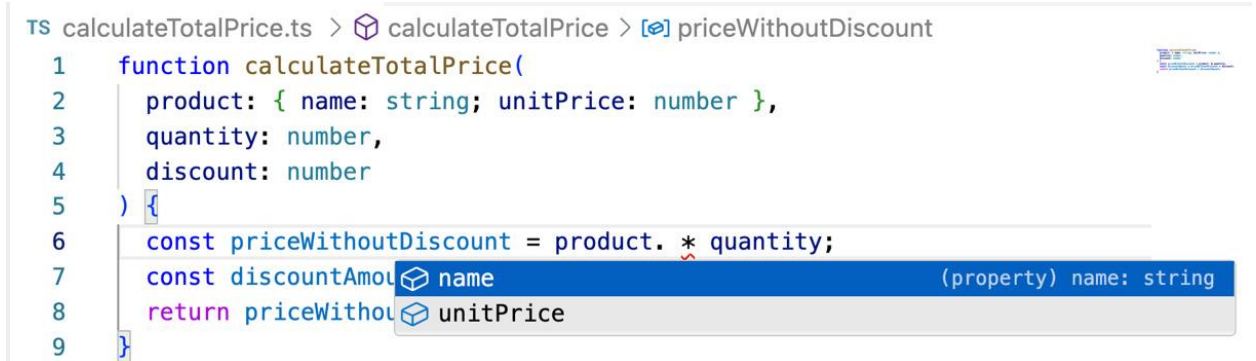


Figura 2.3.4 – IntelliSense en un archivo de TypeScript

Esta vez, Visual Studio Code muestra las propiedades correctas.

- Selecciona **unitPrice** en IntelliSense para resolver el error de tipo.

IntelliSense es solo una de las herramientas que ofrece TypeScript. También proporciona funciones de refactorización sólidas, como renombrar componentes de React, y ayuda a navegar con precisión por el código, por ejemplo, para ir a la definición de una función.

En resumen, en esta sección hemos aprendido lo siguiente:

- La comprobación de tipos de TypeScript ayuda a detectar problemas en una fase más temprana del desarrollo.
- TypeScript permite a los editores de código ofrecer funciones de productividad como IntelliSense.
- Estas ventajas son especialmente importantes al trabajar en bases de código grandes.

Comprender los tipos de JavaScript

Antes de estudiar el sistema de tipos de TypeScript, exploremos brevemente el de JavaScript. Para ello, abre un navegador y sigue estos pasos:

- Abre las herramientas de desarrollo del navegador (**F12** en Windows o **Cmd + Opción + I** en Mac) y ve al panel **Consola**.
- Introduce las siguientes líneas en la consola:

```

let firstName = 'Fred';
console.log(typeof firstName);
let score = 9;
console.log(typeof score);
let date = new Date(2022, 10, 1);
console.log(typeof date);

```

El código asigna distintos valores a tres variables y muestra en la consola el tipo de cada una según JavaScript. Esta es la salida:

```

> let firstName = "Fred";
< undefined
> console.log(typeof firstName)
string VM678:1
< undefined
> let score = 9;
< undefined
> console.log(typeof score);
number VM865:1
< undefined
> let date = new Date(2022, 10, 1);
< undefined
> console.log(typeof date);
object VM904:1
< undefined

```

Figura 2.3.5 – Algunos tipos de JavaScript

No es de extrañar que **firstName** sea **string** y **score** sea **number**. Sin embargo, quizá sorprenda que **date** aparezca como **object**. El objeto **Date** existe en JavaScript, pero el operador **typeof** solo distingue unos pocos tipos primitivos, y para cualquier objeto devuelve **object**. Para comprobar en JavaScript si un valor es una fecha se utiliza **instanceof**:

```
console.log(date instanceof Date); // true
```

3. Añade un par de líneas más de código a la consola:

```
score = 'ten';
console.log(typeof score);
```

Una vez más, la salida de la consola resulta un poco sorprendente:

```

> score = "ten"
< 'ten'
> console.log(typeof score)
string VM1275:1

```

Figura 2.3.6 – Cambio de tipo de variable

¡La variable **score** ha cambiado de tipo **number** a tipo **string**! Esto se debe a que JavaScript es un lenguaje de **tipado dinámico**: el tipo está asociado al valor, no a la variable, por lo que una misma variable puede contener valores de tipos distintos a lo largo de la ejecución.

Nota: tipado dinámico frente a tipado débil

A menudo se confunden estos dos conceptos. El **tipado dinámico** significa que los tipos se comprueban en tiempo de ejecución y que una variable puede cambiar de tipo, como acabamos de ver. El **tipado débil** se refiere a las conversiones implícitas de tipo que realiza el lenguaje, por ejemplo, `'5' * 2` da como resultado `10`. JavaScript tiene ambas características.

Un punto clave es que JavaScript solo tiene un conjunto reducido de tipos, como **string**, **number** y **boolean**. Todos los tipos de JavaScript están disponibles en TypeScript, ya que TypeScript es un superconjunto de JavaScript.

Además, como el motor de JavaScript no genera un error cuando una variable cambia a un tipo completamente diferente, los editores de código no pueden detectar este tipo de errores en JavaScript.

Nota

Para obtener más información sobre los tipos de JavaScript, consulta: Tipos de datos y estructuras en JavaScript – MDN (https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Data_structures).

Ahora que comprendemos las limitaciones del sistema de tipos de JavaScript, aprenderemos sobre el sistema de tipos de TypeScript, comenzando por los tipos básicos.

Uso de los tipos básicos de TypeScript

En esta sección, comenzaremos por comprender cómo se declaran los tipos en TypeScript y cómo se deducen a partir de los valores asignados. A continuación, aprenderemos los tipos básicos que se utilizan habitualmente en TypeScript y veremos casos de uso útiles.

Uso de anotaciones de tipo

Las anotaciones de tipo de TypeScript permiten declarar variables con tipos específicos. Gracias a ellas, el compilador puede comprobar que el código se ajusta a dichos tipos y detectar mucho antes los errores en los que usamos un tipo incorrecto.

Abre TypeScript Playground en <https://www.typescriptlang.org/play> y sigue estos pasos para explorar las anotaciones de tipo:

1. Elimina cualquier código existente en el panel de la izquierda e introduce la siguiente declaración de variable:

```
let unitPrice: number;
```

La anotación de tipo va después del nombre de la variable: dos puntos seguidos del tipo. En este caso, **unitPrice** va a ser de tipo **number**. Recuerda que **number** es un tipo de JavaScript, por lo que también está disponible en TypeScript.

El JavaScript transpilado aparece en el panel de la derecha de la siguiente manera:

```
let unitPrice;
```

Fíjate en que la anotación de tipo ha desaparecido. Esto se debe a que las anotaciones de tipo no existen en JavaScript.

Nota

Es posible que también veas `"use strict";` en la parte superior del JavaScript transpilado. Esto significa que el JavaScript se ejecutará en modo estricto, que detecta más errores de programación. Para obtener más información, consulta https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode.

2. Añade una segunda línea al programa:

```
unitPrice = 'Table';
```

Observa que aparece una línea roja debajo de **unitPrice** en esta línea. Si pasas el cursor por encima, se describe un error de tipo:

```
let unitPrice: number
```

Type 'string' is not assignable to type 'number'. (2322)

[View Problem \(Alt+F8\)](#) No quick fixes available

```
unitPrice = "Table";
```

Figura 2.3.7 – Detección de un error de tipo

El error de tipo también aparece en la pestaña **Errors** del panel de la derecha. Hay una insignia roja con el número 1 que indica que hay un error. Haz clic en esa pestaña para ver el mismo mensaje de error.

.JS .D.TS **Errors** ¹ Logs Plugins

Errors in code

Type 'string' is not assignable to type 'number'.

Figura 2.3.8 – Error de tipo notificado en la pestaña Errors

- También puedes añadir anotaciones de tipo a los parámetros y al valor de retorno de una función utilizando la misma sintaxis. A modo de ejemplo, introduce la siguiente función en TypeScript Playground:

```
function getTotal(
  unitPrice: number,
  quantity: number,
  discount: number,
): number {
  const priceWithoutDiscount = unitPrice * quantity;
  const discountAmount = priceWithoutDiscount * discount;
  return priceWithoutDiscount - discountAmount;
}
```

Hemos declarado los parámetros **unitPrice**, **quantity** y **discount**, todos de tipo **number**. La anotación del tipo de retorno aparece después de los paréntesis de la función y, en este ejemplo, también es **number**.

Nota

Hemos utilizado tanto **const** como **let** para declarar variables. **let** permite que la variable cambie de valor tras la declaración, mientras que una variable **const** no puede reasignarse. En la función anterior, **priceWithoutDiscount** y **discountAmount** nunca cambian de valor tras la asignación inicial, por lo que hemos utilizado **const**.

- Añade otra línea de código que llame a **getTotal** con un tipo incorrecto para **quantity** y asigne el resultado a una variable con un tipo incorrecto:

```
let total: string = getTotal(500, 'one', 0.1);
```

Ambos errores se detectan y resaltan inmediatamente:

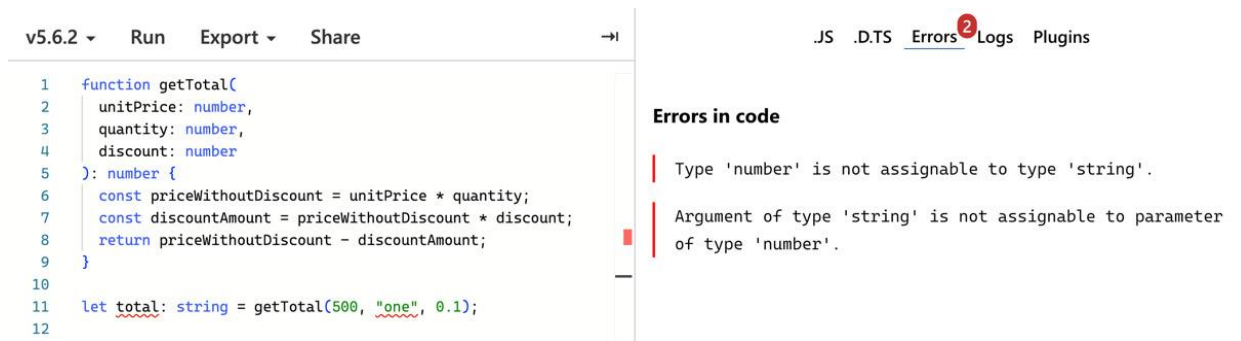


Figura 2.3.9 – Detección de ambos errores de tipo

Esta verificación de tipos tan rigurosa no existe en JavaScript, y resulta muy útil en bases de código grandes porque permite detectar inmediatamente los errores de tipo.

Uso de la inferencia de tipos

Las anotaciones de tipo son muy valiosas, pero requieren escribir código adicional. Por suerte, el sistema de **inferencia de tipos** de TypeScript hace que no sea necesario escribir anotaciones en todo momento: TypeScript infiere el tipo de una variable a partir del valor que se le asigna.

Explora la inferencia de tipos siguiendo estos pasos en TypeScript Playground:

1. Elimina cualquier código anterior y añade la siguiente línea:

```
let flag = false;
```

2. Pasa el cursor por encima de la variable **flag**. Aparecerá una información sobre herramientas que muestra el tipo inferido:

```
let flag: boolean
let flag = false;
```

Figura 2.3.10 – Al pasar el cursor por encima de una variable se muestra su tipo

3. Añade otra línea debajo para asignar a **flag** un valor no válido:

```
flag = 'table';
```

Se detecta inmediatamente un error de tipo, igual que cuando utilizábamos una anotación de tipo.

La inferencia de tipos evita la sobrecarga de código que supondría usar anotaciones en todas partes. Por eso, es una práctica habitual confiar en la inferencia y recurrir a las anotaciones solo cuando la inferencia no proporciona el tipo deseado.

Uso del tipo Date

Date es un objeto global de JavaScript que sirve para trabajar con fechas. Como hemos visto, el operador **typeof** de JavaScript no lo distingue de cualquier otro objeto. TypeScript, en cambio, dispone de un tipo **Date** que representa exactamente ese objeto, por lo que el compilador sabe qué propiedades y métodos tiene una fecha y puede comprobar que los usamos correctamente.

Nota

Consulta el siguiente enlace para obtener más información sobre el objeto **Date** de JavaScript: Date – JavaScript | MDN (https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Date).

Para explorar el tipo **Date** de TypeScript, sigue estos pasos en TypeScript Playground:

1. Elimina cualquier código anterior y añade las siguientes líneas:

```
let today: Date;
today = new Date();
```

Se declara una variable llamada **today** con el tipo **Date** y se le asigna la fecha actual.

2. Reescribe estas dos líneas en una sola utilizando la inferencia de tipos en lugar de una anotación:

```
let today = new Date();
```

3. Comprueba que a **today** se le ha asignado el tipo **Date** pasando el cursor por encima y consultando la información sobre herramientas:

```
let today: Date
let today = new Date();
```

Figura 2.3.11 – Confirmación de que se ha inferido el tipo **Date** para **today**

4. Comprueba que IntelliSense funciona escribiendo **today.** en una nueva línea:

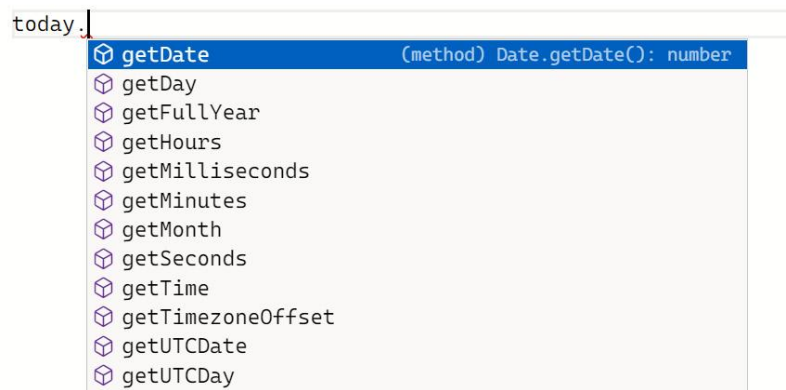


Figura 2.3.12 – IntelliSense funcionando con una fecha

5. Elimina esta línea y añade una línea de código ligeramente diferente:

```
today.addMonths(2);
```

El método **addMonths** no existe en el objeto **Date**, por lo que se produce un error de tipo:



Figura 2.3.13 – Error de tipo detectado en una fecha

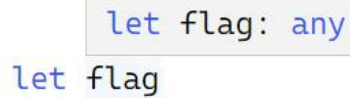
En resumen, el tipo **Date** tiene todas las características que esperamos (inferencia, IntelliSense y comprobación de tipos), que resultan muy útiles al trabajar con fechas.

Uso del tipo any

¿Qué pasa si declaramos una variable sin anotación de tipo y sin valor? ¿Qué tipo inferirá TypeScript? Averigüémoslo introduciendo el siguiente código en TypeScript Playground:

```
let flag;
```

Ahora, pasa el ratón por encima de **flag**:



```
let flag: any
```

Figura 2.3.14 – Variable con el tipo any

TypeScript asigna el tipo **any** a una variable sin anotación de tipo y sin valor inicial. **any** desactiva la comprobación de tipos para esa variable: se le puede asignar cualquier valor y se puede usar de cualquier manera sin que el compilador proteste. Se ha usado habitualmente para contenido dinámico o para valores procedentes de bibliotecas de terceros, pero el sistema de tipos cada vez más potente de TypeScript hace que hoy en día sea necesario con mucha menos frecuencia.

Existe una alternativa mejor: el tipo **unknown**.

Uso del tipo unknown

unknown es un tipo que podemos utilizar cuando no sabemos qué tipo tendrá un valor, pero queremos trabajar con él de forma segura. Sigue estos pasos para descubrir por qué es una alternativa mejor que **any**:

1. En TypeScript Playground, elimina cualquier código anterior e introduce lo siguiente:

```
fetch('https://swapi.dev/api/people/1')
  .then((response) => response.json())
  .then((data) => {
    console.log('firstName', data.firstName);
  });
```

El código recupera un personaje de Star Wars desde una API web. No se producen errores de tipo, por lo que el código parece correcto.

Nota

El servicio **swapi.dev** ha sufrido caídas con cierta frecuencia. Si no responde, puedes usar cualquier otra API pública que devuelva un objeto con una propiedad **name**, por ejemplo, <https://jsonplaceholder.typicode.com/users/1>.

2. Haz clic en la opción **Run** para ejecutar el código:

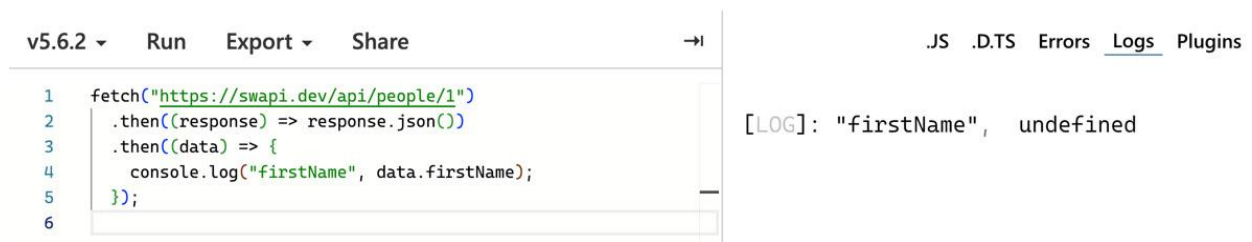


Figura 2.3.15 – La propiedad firstName tiene un valor undefined

La propiedad **firstName** no existe en los datos recuperados, ya que se muestra **undefined** en la consola.

¿Por qué no se produjo un error de tipo en la línea 4, donde se hace referencia a **firstName**? Porque **data** es de tipo **any**, lo que significa que no se realiza ninguna comprobación de tipos sobre ella. Puedes pasar el cursor por encima de **data** para confirmarlo.

3. Asigna a **data** la anotación de tipo **unknown**:

```
fetch('https://swapi.dev/api/people/1')
  .then((response) => response.json())
  .then((data: unknown) => {
    console.log('firstName', data.firstName);
  });
```

Ahora se produce un error de tipo donde se hace referencia a **firstName**:



Figura 2.3.16 – Error de tipo en el parámetro data de tipo unknown

unknown es el tipo más general de TypeScript: igual que **any**, admite cualquier valor. La diferencia es que **unknown** no permite hacer nada con ese valor (acceder a propiedades, llamarlo como función, operar con él...) hasta que comprobemos qué es. Esas comprobaciones permiten a TypeScript **estrechar** el tipo (en inglés, *type narrowing*), es decir, pasar de un tipo general a otro más concreto.

4. Antes de proporcionar a TypeScript la información necesaria para estrechar el tipo de **data**, cambia la propiedad a la que se hace referencia de **firstName** a **name**:

```
fetch('https://swapi.dev/api/people/1')
  .then((response) => response.json())
  .then((data: unknown) => {
    console.log('name', data.name);
  });
```

name es una propiedad válida, pero sigue produciéndose un error de tipo. Esto se debe a que **data** sigue siendo **unknown**.

5. Ahora, realiza los siguientes cambios para estrechar el tipo de **data**:

```
fetch('https://swapi.dev/api/people/1')
  .then((response) => response.json())
  .then((data: unknown) => {
    if (isCharacter(data)) {
      console.log('name', data.name);
    }
  });

function isCharacter(
  character: unknown,
): character is { name: string } {
  return (
    typeof character === 'object' &&
```

```

    character !== null &&
    'name' in character &&
    typeof character.name === 'string'
  );
}

```

La instrucción **if** utiliza una función llamada **isCharacter** para verificar que el valor es un objeto con una propiedad **name** de tipo **string**. Fíjate en que el parámetro de **isCharacter** también es de tipo **unknown**: cada comprobación de la expresión estrecha un poco más el tipo. Primero se comprueba que es un objeto, después que no es **null** (porque **typeof null** también devuelve **object**), luego que tiene la propiedad **name** y, por último, que esa propiedad es una cadena.

Fíjate en el tipo de retorno de **isCharacter**:

```
character is { name: string }
```

Se trata de un **predicado de tipo**. Si la función devuelve **true**, TypeScript estrecha el tipo del argumento a **{ name: string }**. A este tipo de funciones se las llama **guardas de tipo** (*type guards*).

6. Pasa el cursor por encima de **data** en cada línea donde aparece. **data** comienza con el tipo **unknown** y, dentro de la rama **if**, se estrecha a **{ name: string }**:

```

fetch("https://swapi.dev/api/people/1")
  .then((response) => resp)
  .then((data: unknown) => {
    if (isCharacter(data)) {
      console.log("name", data.name);
    }
  });

```

Figura 2.3.17 – Tipo estrechado de data dentro de la rama if

Fíjate en que el error de tipo también ha desaparecido. ¡Genial!

7. Ejecuta el código. Verás que se muestra «Luke Skywalker» en la consola.

En resumen, **unknown** es una excelente opción para datos cuyo tipo no conoces con certeza. Sin embargo, no puedes interactuar con una variable **unknown** hasta haber estrechado su tipo mediante comprobaciones.

Nota: el tipo never

No hay que confundir **unknown** con **never**. **unknown** admite cualquier valor; **never** es justo lo contrario: un tipo que no admite ningún valor. Se utiliza, por ejemplo, para funciones que nunca terminan normalmente:

```
function lanzarError(mensaje: string): never { throw new Error(mensaje); }
```

Uso de arrays

Los arrays son estructuras que TypeScript hereda de JavaScript. Se les añaden anotaciones de tipo como de costumbre, pero con corchetes **[]** al final para indicar que se trata de un array.

Veamos un ejemplo en TypeScript Playground:

1. Elimina cualquier código existente e introduce lo siguiente:

```
const numbers: number[] = [];
```

Como alternativa, se puede utilizar la sintaxis del tipo genérico **Array**:

```
const numbers: Array<number> = [];
```

2. Añade **1** al array utilizando el método **push**:

```
numbers.push(1);
```

3. Ahora añade una cadena al array:

```
numbers.push('two');
```

Se produce un error de tipo, tal y como cabría esperar:

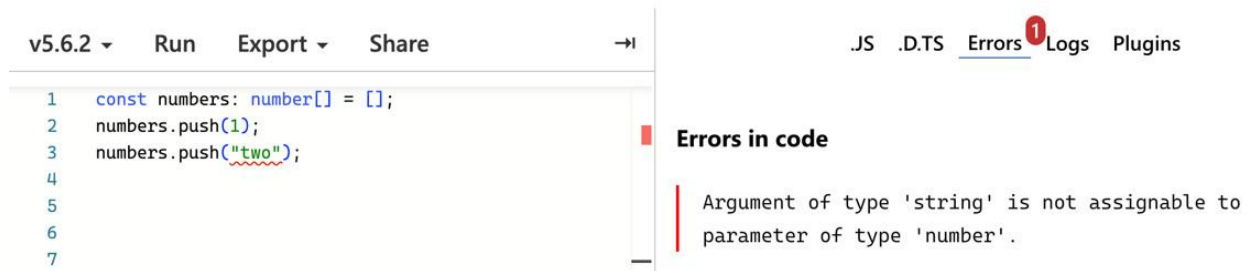


Figura 2.3.18 – Error de tipo al añadir una cadena a un array de números

4. Ahora sustituye todo el código por lo siguiente:

```
const numbers = [1, 2, 3];
```

Pasa el cursor por encima de **numbers** para comprobar que TypeScript ha deducido que su tipo es **number[]**.

```
const numbers: number[]  
const numbers = [1, 2, 3];
```

Figura 2.3.19 – Inferencia del tipo de un array

¡Podemos ver que la inferencia de tipos de TypeScript también funciona con los arrays!

Los arrays son una de las estructuras más utilizadas. En los ejemplos anteriores solo hemos usado arrays de números, pero se puede utilizar cualquier tipo para los elementos, incluidos los objetos.

A continuación, un resumen de los tipos básicos que hemos aprendido en esta sección:

- TypeScript permite usar todos los tipos de JavaScript y conoce con precisión objetos como **Date**, lo que permite comprobar su uso.
- TypeScript puede inferir el tipo de una variable a partir del valor que se le asigna. Se puede utilizar una anotación de tipo cuando la inferencia no proporciona el tipo deseado.
- No se realiza ninguna comprobación de tipos en las variables de tipo **any**, por lo que este tipo debe evitarse.
- El tipo **unknown** es una alternativa segura a **any**: admite cualquier valor, pero hay que estrechar su tipo antes de poder interactuar con él.
- Los tipos de array se definen con corchetes después del tipo de los elementos (o con la sintaxis **Array<T>**).

Creación de tipos en TypeScript

En la sección anterior vimos que TypeScript cuenta con un amplio conjunto de tipos estándar. En esta sección, aprenderemos a crear nuestros propios tipos: primero para objetos y después para variables que pueden contener un conjunto concreto de valores.

Uso de tipos de objeto

Los objetos son muy comunes en los programas de JavaScript, por lo que es muy importante aprender a representarlos en TypeScript. De hecho, ya hemos utilizado un tipo de objeto en este tema para el parámetro **product** de la función **calculateTotalPrice**:

```
function calculateTotalPrice(
  product: { name: string; unitPrice: number },
  ...
) {
  ...
}
```

Un tipo de objeto en TypeScript se escribe de forma similar a un objeto literal de JavaScript, pero en lugar de valores se especifican los tipos de las propiedades. Las propiedades pueden separarse con punto y coma o con comas, aunque lo habitual es usar punto y coma.

Borra cualquier código existente en TypeScript Playground y sigue este ejemplo para explorar los tipos de objeto:

1. Introduce la siguiente asignación de una variable a un objeto:

```
let table = { name: 'Table', unitPrice: 450 };
```

Si pasas el cursor por encima de la variable **table**, verás que se infiere el siguiente tipo:

```
{
  name: string;
  unitPrice: number;
}
```

Así pues, la inferencia de tipos funciona bien con los objetos.

2. En la siguiente línea, intenta establecer la propiedad **discount** en **10**:

```
table.discount = 10;
```

La propiedad **discount** no existe en el tipo; solo existen **name** y **unitPrice**. Por lo tanto, se produce un error de tipo.

3. Supongamos que queremos representar un objeto de producto con las propiedades **name** y **unitPrice**, pero queremos que **unitPrice** sea opcional. Elimina el código existente y sustitúyelo por lo siguiente:

```
const table: { name: string; unitPrice: number } = {
  name: 'Table',
};
```

Esto genera un error de tipo porque **unitPrice** es una propiedad obligatoria en la anotación de tipo.

4. Podemos usar el símbolo **?** para que la propiedad sea opcional en lugar de obligatoria:

```
const table: { name: string; unitPrice?: number } = {
```

```
name: 'Table',
};
```

El error de tipo desaparece.

Nota

El símbolo `?` también se puede utilizar en funciones para indicar parámetros opcionales, por ejemplo: `myFunction(requiredParam: string, optionalParam?: string)`.

Ahora, veamos cómo simplificar las definiciones de tipos de objetos.

Creación de alias de tipo

La anotación de tipo del último ejemplo era bastante larga, y lo sería aún más con estructuras de objetos más complejas. Además, tener que repetir la misma estructura para cada variable resulta tedioso:

```
const table: { name: string; unitPrice?: number } = ...;
const chair: { name: string; unitPrice?: number } = ...;
```

Los **alias de tipo** resuelven estos problemas. Como su nombre indica, un alias de tipo da un nombre a otro tipo. La sintaxis es la siguiente:

```
type YourTypeAliasName = AnExistingType;
```

Abre TypeScript Playground y sigue estos pasos para explorar los alias de tipo:

1. Empieza creando un alias de tipo para la estructura del objeto **product** que utilizamos en el último ejemplo:

```
type Product = { name: string; unitPrice?: number };
```

2. Ahora asigna este tipo **Product** a dos variables:

```
let table: Product = { name: 'Table' };
let chair: Product = { name: 'Chair', unitPrice: 40 };
```

¡Así queda mucho más claro!

3. Un alias de tipo puede extender otro tipo utilizando el símbolo **&**. Crea un segundo tipo para un producto con descuento:

```
type DiscountedProduct = Product & { discount: number };
```

DiscountedProduct representa un objeto que contiene las propiedades **name**, **unitPrice** (opcional) y **discount**.

Nota

Un tipo creado combinando otros con el símbolo **&** se denomina **tipo de intersección**.

4. Añade la siguiente variable con el tipo **DiscountedProduct**:

```
let chairOnSale: DiscountedProduct = {
  name: 'Chair on Sale',
  unitPrice: 30,
  discount: 5,
};
```

5. También se puede utilizar un alias de tipo para representar una función. Añade el siguiente alias:

```
type Purchase = (quantity: number) => void;
```

Este tipo representa una función que recibe un parámetro de tipo **number** y no devuelve nada.

Nota

El tipo **void** se utiliza para indicar que una función no devuelve ningún valor.

6. Utiliza el tipo **Purchase** para crear una propiedad de función **purchase** en el tipo **Product**:

```
type Purchase = (quantity: number) => void;
type Product = {
  name: string;
  unitPrice?: number;
  purchase: Purchase;
};
```

Se producirán errores de tipo en las declaraciones de las variables **table**, **chair** y **chairOnSale**, ya que la propiedad **purchase** es obligatoria.

7. Añade una propiedad de función **purchase** a la declaración de la variable **table**:

```
let table: Product = {
  name: 'Table',
  purchase: (quantity) =>
    console.log(`Purchased ${quantity} tables`),
};
table.purchase(4);
```

El error de tipo se resuelve en la declaración de **table**. Fíjate en que no ha sido necesario anotar el tipo del parámetro **quantity**: TypeScript lo infiere a partir del tipo **Purchase**.

Se podría añadir una propiedad **purchase** de forma similar a **chair** y **chairOnSale** para resolver sus errores. Sin embargo, ignóralos para esta exploración y pasa al siguiente paso.

8. Haz clic en la opción **Run** para ejecutar el código que compra cuatro mesas. Se muestra el mensaje **Purchased 4 tables** en la consola.

En resumen, los alias de tipo permiten combinar tipos existentes y mejoran la legibilidad y la reutilización de los tipos. En este tema y en los siguientes los utilizaremos de forma habitual.

Creación de tipos de unión

Un **tipo de unión** combina varios tipos en uno nuevo que puede representar cualquiera de ellos. Los alias de tipo se pueden utilizar para dar nombre a tipos de unión. Un ejemplo:

```
type Level = 'H' | 'M' | 'L';
```

Una variable de tipo **Level** solo puede contener los valores **'H'**, **'M'** o **'L'**.

Borra cualquier código existente en TypeScript Playground y probemos los tipos de unión:

1. Empieza creando un tipo que represente **'red'**, **'green'** o **'blue'**:

```
type RGB = 'red' | 'green' | 'blue';
```

Este tipo es una unión de cadenas literales, pero un tipo de unión puede estar formado por cualquier tipo, ¡incluso tipos mezclados!

2. Crea una variable con el tipo **RGB** y asígnale un valor válido:

```
let color: RGB = 'red';
```

3. Ahora intenta asignar un valor que no pertenezca al tipo:

```
color = 'yellow';
```

Se produce un error de tipo, como era de esperar:

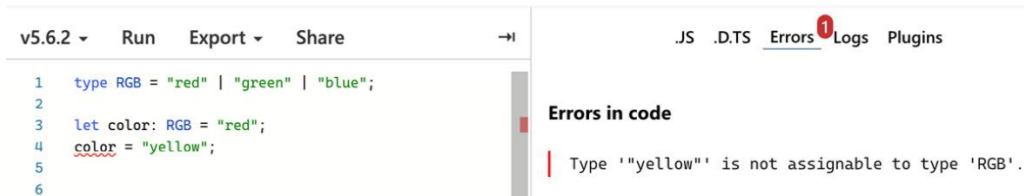


Figura 2.3.20 – Error de tipo en un tipo de unión

4. Ahora intenta asignar **null** a **color**:

```
color = null;
```

Esto también genera un error.

5. Los tipos de unión pueden combinar tipos diferentes. Añade **null** al tipo **RGB**:

```
type RGB = 'red' | 'green' | 'blue' | null;
```

La asignación de **null** a **color** ya no genera un error, porque **null** forma parte del tipo **RGB**.

Interfaces frente a alias de tipo

En el tema UT 01 se presentaron las **interfaces**. Como verás ambas formas en la documentación de React y en proyectos reales, conviene conocer sus diferencias. Para describir la forma de un objeto, una interfaz y un alias de tipo son prácticamente equivalentes:

```
interface Alumno {
  nombre: string;
  curso: number;
}

type Profesor = {
  nombre: string;
  asignaturas: string[];
};
```

Las diferencias principales son las siguientes:

Característica	interface	type
Describir la forma de un objeto	Sí	Sí
Ampliar otro tipo	Con extends	Con & (intersección)
Tipos de unión ('a' 'b')	No	Sí
Dar nombre a primitivos, tuplas o funciones	No (salvo funciones, con sintaxis menos clara)	Sí
Declaraciones repetidas que se fusionan	Sí	No (da error)

Así se amplía un tipo con cada sintaxis:

```
interface AlumnoErasmus extends Alumno {
  paisOrigen: string;
}

type ProfesorTutor = Profesor & { grupoTutorizado: string };
```

En proyectos de React es habitual usar **type** para las props de los componentes, porque permite combinarlas fácilmente con uniones e intersecciones. Lo importante es ser coherente dentro de un mismo proyecto.

Objetos constantes con `as const` y `satisfies`

Dos herramientas muy utilizadas en el código TypeScript actual son **as const** y **satisfies**.

as const indica a TypeScript que un valor no va a cambiar, por lo que infiere los tipos más concretos posibles (los literales) y marca las propiedades como de solo lectura:

```
const configuracion = { tema: 'oscuro', idioma: 'es' };
// tipo inferido: { tema: string; idioma: string }

const configuracionFija = { tema: 'oscuro', idioma: 'es' } as const;
// tipo inferido: { readonly tema: 'oscuro'; readonly idioma: 'es' }
```

satisfies comprueba que un valor cumple un tipo sin perder el tipo concreto que se ha inferido:

```
type Tema = 'claro' | 'oscuro';

const ajustes = {
  tema: 'oscuro',
  tamanoFuente: 16,
} satisfies { tema: Tema; tamanoFuente: number };

ajustes.tema; // tipo: 'oscuro' (no se pierde el literal)
```

Combinando **as const** con un tipo derivado se obtiene la alternativa recomendada hoy a los **enum** del tema UT 01:

```
const Rol = {
  Admin: 'ADMIN',
  SoloLectura: 'SOLO_LECTURA',
  Autor: 'AUTOR',
} as const;

type Rol = (typeof Rol)[keyof typeof Rol];
// 'ADMIN' | 'SOLO_LECTURA' | 'AUTOR'

const miRol: Rol = Rol.Admin;
```

Novedad: ¿por qué evitar `enum`?

Los **enum** son una de las pocas características de TypeScript que generan código JavaScript propio, en lugar de limitarse a desaparecer al compilar. Las herramientas modernas (Vite, Node.js al ejecutar archivos .ts directamente) prefieren que el código TypeScript se pueda convertir a JavaScript simplemente **borrando** los tipos. La opción **erasableSyntaxOnly** del compilador, activada en la plantilla de Vite que usaremos más adelante, prohíbe los **enum** por este motivo.

A continuación, un resumen de lo que hemos aprendido sobre la creación de tipos:

- Los tipos de unión son una forma estupenda de representar un conjunto concreto de valores literales. También permiten que una variable contenga valores de varios tipos.
- Los alias de tipo permiten crear tipos reutilizables y se pueden usar para objetos, funciones y tipos de unión.
- Un alias de tipo existente se puede ampliar con el símbolo **&**; una interfaz, con **extends**.
- El símbolo **?** indica que una propiedad de un objeto o un parámetro de una función es opcional.
- **as const** y **satisfies** permiten obtener tipos más precisos; juntos ofrecen una alternativa moderna a los **enum**.

Uso del compilador de TypeScript

En esta sección, aprenderemos a utilizar el compilador de TypeScript para comprobar los tipos del código y transpilarlo a JavaScript. Crearemos con Visual Studio Code un proyecto sencillo de TypeScript que contenga una función y usaremos el terminal de Visual Studio Code para interactuar con el compilador.

Abre Visual Studio Code en una carpeta vacía de tu elección y sigue estos pasos:

1. En el panel **Explorador**, crea un archivo llamado **package.json** con el siguiente contenido:

```
{
  "name": "tsc-play",
  "dependencies": {
    "typescript": "*"
  },
  "scripts": {
    "build": "tsc src/welcome.ts"
  }
}
```

El archivo define un nombre de proyecto, **tsc-play**, y establece la última versión de TypeScript como única dependencia. También define un script de npm llamado **build**, que invocará el compilador de TypeScript (**tsc**) pasándole el archivo **welcome.ts** de la carpeta **src**. No te preocupes si **welcome.ts** no existe todavía: lo crearemos enseguida.

2. En un terminal, ve a la carpeta del proyecto e instala las dependencias con el siguiente comando:

```
npm i
```

Esto instalará todas las bibliotecas de la sección **dependencies** del archivo **package.json**; en este caso, TypeScript.

Nota

Con **"typescript": "*" se instala la versión más reciente, que actualmente es TypeScript 7 (el compilador nativo). Para este ejercicio no hay ninguna diferencia. Si en algún momento necesitas una versión concreta, puedes indicarla, por ejemplo: `npm i -D typescript@6`.**

3. Crea una carpeta llamada **src** y, dentro de ella, un archivo llamado **welcome.ts**.
4. Abre **welcome.ts** y añade el siguiente contenido:

```
function welcome(name: string | null) {
  if (name === null) {
    return `¡Bienvenido!`;
  }
}
```

```

    }
    return `¡Bienvenido, ${name}!`;
  }
}

```

La función recibe un nombre y construye una cadena que le da la bienvenida. Si no hay nombre, se devuelve un mensaje de bienvenida genérico.

- Introduce el siguiente comando en el terminal:

```
npm run build
```

Esto ejecutará el script **build** que definimos en el primer paso.

- Una vez finalizado el comando, fíjate en que aparece un archivo **welcome.js** junto a **welcome.ts** en la carpeta **src**.
- Abre el archivo **welcome.js** transpilado. Tendrá un aspecto similar a este:

```

function welcome(name) {
  if (name === null) {
    return `¡Bienvenido!`;
  }
  return `¡Bienvenido, ${name}!`;
}

```

Fíjate en que se han eliminado las anotaciones de tipo, porque no son válidas en JavaScript.

Novedad (TypeScript 6 y 7)

En versiones anteriores a TypeScript 6, el compilador transpilaba por defecto a una versión antigua de JavaScript (ES5) y convertía la plantilla literal en algo como `"¡Bienvenido, ".concat(name, "!"`). Desde TypeScript 6, la versión de destino por defecto es la más reciente de ECMAScript, por lo que el código generado conserva las características modernas.

- Aun así, la configuración por defecto del compilador no es la ideal: probablemente queramos que el JavaScript transpilado esté en una carpeta separada y controlar exactamente qué archivos se compilan. El compilador se configura mediante un archivo llamado **tsconfig.json**. Añade un archivo **tsconfig.json** en la raíz del proyecto con el siguiente contenido:

```

{
  "compilerOptions": {
    "outDir": "dist",
    "target": "esnext",
    "module": "esnext",
    "lib": ["DOM", "esnext"],
    "strict": true,
    "jsx": "react-jsx",
    "moduleResolution": "bundler",
    "noEmitOnError": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}

```

A continuación se explica cada opción del campo **compilerOptions**:

Opción	Significado
outDir	Carpeta en la que se coloca el JavaScript transpilado.
target	Versión de JavaScript a la que se transpila. esnext significa la versión más reciente: se usan las características modernas de JavaScript en lugar de

Opción	Significado
	reemplazarlas por código equivalente (<i>polyfills</i>), lo que reduce el tamaño del código generado.
module	Sistema de módulos del código. esnext hace referencia a los módulos estándar de JavaScript (import/export).
lib	Tipos de la biblioteca estándar incluidos en la comprobación. DOM proporciona los tipos de la API del navegador y esnext , los de las API de JavaScript más recientes.
strict	Aplica el nivel más estricto de comprobación de tipos. Incluye, entre otras, strictNullChecks y noImplicitAny . Desde TypeScript 6 ya está activado por defecto, pero es buena práctica indicarlo explícitamente.
jsx	Indica cómo se transpila el JSX de React. react-jsx es el modo actual, que no obliga a importar React en cada archivo.
moduleResolution	Forma de buscar las dependencias importadas. bundler es el modo recomendado cuando se trabaja con herramientas como Vite.
noEmitOnError	Si es true , no se genera JavaScript cuando hay errores de tipo.

El campo **include** especifica los archivos TypeScript que se van a compilar, y el campo **exclude**, los que se excluyen.

Por ejemplo, con **strict** activado, la siguiente instrucción da error, porque **null** no forma parte del tipo **string**:

```
let firstName: string = null;
```

Nota: opciones obsoletas

Es posible que encuentres en tutoriales antiguos **"moduleResolution": "node"** o **"jsx": "react"**. La resolución **node** quedó obsoleta en TypeScript 6 y da error en TypeScript 7; usa **bundler**, **node16** o **nodenext** en su lugar. Para obtener más información sobre las opciones del compilador, consulta <https://www.typescriptlang.org/tsconfig>.

- La configuración del compilador ya especifica que se compilen todos los archivos de la carpeta **src**. Por lo tanto, elimina la ruta del archivo en el script **build** de **package.json**:

```
{
  ...,
  "scripts": {
    "build": "tsc"
  }
}
```

Nota

Si se pasa un archivo concreto al comando **tsc** (como en **tsc src/welcome.ts**), el compilador **ignora** el archivo **tsconfig.json**. Por eso es necesario este cambio.

- Elimina el archivo **welcome.js** transpilado anteriormente de la carpeta **src**.
- Vuelve a ejecutar el comando **build** en el terminal:

```
npm run build
```

Esta vez, el archivo transpilado se coloca en una carpeta **dist**.

- Por último, provocaremos un error de tipo. Abre **welcome.ts** y añade un tipo de retorno **number** a la función:

```
function welcome(name: string | null): number {
```

El editor muestra inmediatamente dos errores de tipo.

- Elimina la carpeta **dist** para borrar el archivo JavaScript transpilado anteriormente.
- Vuelve a ejecutar el comando **build** en el terminal:

```
npm run build
```

Los errores de tipo se muestran en el terminal. Fíjate en que no se crea el archivo JavaScript transpilado, gracias a la opción **noEmitOnError**.

Novedad: ejecutar TypeScript directamente con Node.js

Las versiones actuales de Node.js (22.18 y posteriores) pueden ejecutar archivos **.ts** directamente, sin compilarlos antes: **node src/welcome.ts**. Para ello, Node.js simplemente borra las anotaciones de tipo. Es importante tener en cuenta que **Node.js no comprueba los tipos**: para detectar errores de tipo sigue siendo necesario el compilador (**tsc** o **tsc --noEmit**, que comprueba sin generar archivos).

Creación de un componente de React y TypeScript

En los temas UT 02.1 y UT 02.2, «Introducción a React», creamos un componente de alerta con React. En esta sección, utilizaremos TypeScript para que el componente sea fuertemente tipado y experimentaremos sus ventajas. Comenzaremos añadiendo un tipo a las props del componente y, a continuación, experimentaremos con la definición del tipo de su estado.

Creación de un proyecto

Usaremos Vite para crear el proyecto, tal y como hicimos al crear por primera vez el componente de alerta. Sin embargo, esta vez elegiremos la plantilla de React con TypeScript. Sigue estos pasos:

- En un terminal, dentro de la carpeta que elijas, ejecuta el siguiente comando para indicar a Vite que cree un proyecto de React y TypeScript:

```
npm create vite@latest alert -- --template react-ts
```

Hemos especificado el nombre del proyecto y la plantilla en el comando, por lo que no se nos pedirá esta información. La plantilla **react-ts** crea un proyecto de React y TypeScript. Según la versión de Vite, el asistente puede preguntar además si quieres instalar las dependencias y arrancar el proyecto inmediatamente; responde que no para seguir los pasos de este tema.

- Ejecuta los siguientes comandos en el terminal para cambiar a la carpeta **alert**, instalar las dependencias, abrir el proyecto en Visual Studio Code y ejecutar la aplicación en modo de desarrollo:

```
cd alert
npm i
code .
npm run dev
```

Novedad: Vite 8 y los archivos tsconfig de la plantilla

Desde marzo de 2026, Vite 8 utiliza **Rollup**, un empaquetador escrito en Rust, lo que acelera notablemente las compilaciones. Los comandos que usamos no cambian.

Fíjate en que la plantilla genera **tres** archivos de configuración de TypeScript: **tsconfig.json**, que solo hace referencia a los otros dos; **tsconfig.app.json**, con la configuración del código de la aplicación (carpeta **src**), y **tsconfig.node.json**, con la configuración de **vite.config.ts**, que se ejecuta en Node.js.

En **tsconfig.app.json** merece la pena fijarse en dos opciones: **verbatimModuleSyntax**, que obliga a importar los tipos con la palabra clave **type** (lo veremos enseguida), y **erasableSyntaxOnly**, que prohíbe la sintaxis de TypeScript que no se puede borrar sin más, como los **enum**.

3. Crea un nuevo archivo para el componente de alerta en la carpeta **src** llamado **Alert.tsx**.

Nota

Los componentes de React escritos en TypeScript tienen la extensión de archivo **.tsx**.

4. Pega en **Alert.tsx** la versión en JavaScript del componente de alerta del tema UT 02.2.

Fíjate en que se notifican errores de tipo en las props. Al no tener anotación de tipo, TypeScript solo puede asignarles el tipo **any** implícitamente, y la opción **strict** (en concreto, **noImplicitAny**) lo considera un error.

A continuación, añadiremos una anotación de tipo a las props del componente, lo que resolverá estos errores.

Añadir un tipo para las props

Anteriormente en este tema aprendimos a añadir anotaciones de tipo a las funciones. Usaremos este conocimiento para tipar de forma estricta las props del componente de alerta:

1. Añade el siguiente tipo justo encima del componente. Será el tipo de las props del componente:

```
type Props = {
  type?: 'information' | 'warning';
  heading: string;
  children: ReactNode;
  closable?: boolean;
  onClose?: () => void;
};
```

Las props **heading** y **children** son obligatorias; el resto son opcionales.

Fíjate en que la prop **type** no es de tipo **string**, sino un **tipo de unión** con los dos únicos valores que entiende el componente. Si usáramos **string**, el componente aceptaría cualquier cadena, incluida una errata como **'warnig'**, y el error pasaría desapercibido. Con la unión, el editor autocompleta los valores válidos y marca como error cualquier otro.

A la prop **children** se le asigna un tipo especial de React llamado **ReactNode**, que le permite aceptar elementos JSX, cadenas, números, etc. Por el momento, se produce un error en **ReactNode**; lo resolveremos en el siguiente paso.

El nombre del tipo puede ser cualquiera, pero es habitual llamarlo **Props**.

2. Para resolver el error en la referencia a **ReactNode**, actualiza la instrucción **import** de React de la siguiente manera:

```
import { useState, type ReactNode } from 'react';
```

Nota: importar tipos con la palabra clave type

En este proyecto, la palabra clave **type** antes de **ReactNode** es **obligatoria**. La plantilla de Vite activa la opción **verbatimModuleSyntax**, que exige marcar con **type** todo lo que se importa y que solo es un tipo. Así, al convertir el código a JavaScript, basta con borrar esas importaciones,

ya que los tipos no existen en tiempo de ejecución. Si se omite, aparece un error similar a: *'ReactNode' is a type and must be imported using a type-only import when 'verbatimModuleSyntax' is enabled.*

En proyectos sin esa opción no es obligatorio, pero sigue siendo una buena práctica.

3. Ahora, asigna el tipo **Props** al componente de alerta después de los parámetros desestructurados:

```
export function Alert({
  type = 'information',
  heading,
  children,
  closable,
  onClose,
}: Props) {
  ...
}
```

Las props de la alerta ya están fuertemente tipadas. Este es el componente completo:

```
import { useState, type ReactNode } from 'react';

type Props = {
  type?: 'information' | 'warning';
  heading: string;
  children: ReactNode;
  closable?: boolean;
  onClose?: () => void;
};

export function Alert({
  type = 'information',
  heading,
  children,
  closable,
  onClose,
}: Props) {
  const [visible, setVisible] = useState(true);

  if (!visible) {
    return null;
  }

  function handleCloseClick() {
    setVisible(false);
    if (onClose) {
      onClose();
    }
  }

  return (
    <div>
      <div>
        <span
          role="img"
          aria-label={type === 'warning' ? 'Advertencia' : 'Información'}
        >
          {type === 'warning' ? '⚠️' : 'ℹ️'}
        </span>
```

```

    <span>{heading}</span>
  </div>
  {closable && (
    <button aria-label="Cerrar" onClick={handleCloseClick}>
      <span role="img" aria-label="Cerrar">
        ✖
      </span>
    </button>
  )}
  <div>{children}</div>
</div>
);
}

```

Nota: el operador de encadenamiento opcional

Las líneas `if (onClose) { onClose(); }` se pueden abreviar como `onClose?.()`; El operador `?` solo llama a la función si no es **undefined** ni **null**.

Nota: ¿y React.FC?

En tutoriales antiguos verás componentes declarados como `const Alert: React.FC<Props> = (... ) => {...}`. Hoy no se recomienda: basta con tipar las props directamente, como hemos hecho.

4. Abre **App.tsx** y sustituye su contenido por lo siguiente:

```

import { Alert } from './Alert';
import './App.css';

function App() {
  return <Alert />;
}

export default App;

```

Hemos importado el componente **Alert** y lo hemos mostrado en el componente **App**. Se produce un error de tipo en **Alert** dentro del JSX porque no se han pasado las props obligatorias.

```

function App() {
  return <Alert />;
}

export def

```

Type '{}' is missing the following properties from type 'Props': heading, children ts(2739)

(alias) function Alert({ type, heading, children, closable, onClose }: Props): JSX.Element | null

import Alert

View Problem (⌘F8) Quick Fix... (⌘.)

Figura 2.3.21 – Error de tipo en el componente Alert

5. Pasa la prop **heading** a **Alert** y añádele algún contenido:

```

<Alert heading="Success">
  ¡Todo va muy bien!
</Alert>

```

Los errores de tipo desaparecen, ya que se han pasado las props obligatorias.

6. Prueba ahora a pasar un valor no válido en la prop **type**:

```
<Alert type="error" heading="Fallo">
  Algo ha ido mal
</Alert>
```

Se produce un error de tipo, porque **'error'** no forma parte de la unión **'information' | 'warning'**. Corrígelo usando **type="warning"**.

7. Inicia la aplicación ejecutando **npm run dev** en un terminal, si no se está ejecutando.
8. Visita la aplicación en ejecución en un navegador: la alerta aparece en la página tal y como se esperaba.

Añadir un tipo para el estado

Sigue estos pasos para experimentar con el tipo del estado **visible** del componente de alerta:

1. Abre **Alert.tsx** y pasa el cursor por encima de la variable de estado **visible** para ver su tipo inferido. Se ha inferido **boolean**, porque se ha inicializado con el valor **true**. Es justo el tipo que queremos.
2. A modo de experimento, elimina el valor inicial **true** que se pasa a **useState**. Vuelve a pasar el cursor sobre **visible**: ahora se infiere **undefined**, porque no se ha pasado ningún valor inicial. Obviamente, no es el tipo que queremos.
3. Cuando el tipo inferido por **useState** no es el deseado, se puede indicar explícitamente mediante un **argumento genérico**. Añade el siguiente argumento genérico:

```
const [visible, setVisible] = useState<boolean>();
```

Pasa el cursor sobre **visible**: su tipo es ahora **boolean | undefined**. TypeScript añade **undefined** porque, al no haber valor inicial, el estado empieza valiendo **undefined**. Si quisiéramos un tipo **boolean** sin más, deberíamos pasar también el valor inicial: **useState<boolean>(true)**.

Nota

Un argumento genérico es como un argumento de función normal, pero en lugar de un valor define un tipo. Se escribe entre corchetes angulares (**< >**) después del nombre de la función.

4. Restablece la instrucción **useState** a su estado original, inicializada con **true** y sin tipo explícito:

```
const [visible, setVisible] = useState(true);
```

5. Detén la ejecución de la aplicación pulsando **Ctrl + C**.

En resumen, comprueba siempre el tipo de estado inferido por **useState** y utiliza su argumento genérico para indicar el tipo explícitamente cuando el inferido no sea el adecuado. El caso más habitual es un estado que empieza vacío y luego recibe datos, por ejemplo:

```
type Usuario = { nombre: string; correo: string };

const [usuario, setUsuario] = useState<Usuario | null>(null);
```

Tipar los eventos

En el tema UT 02.2 aprendimos a gestionar eventos. Cuando el manejador se escribe directamente en el JSX, TypeScript infiere el tipo del evento automáticamente. Sin embargo, si lo extraemos a una función independiente, debemos indicar su tipo. React proporciona tipos para cada evento, como **ChangeEvent**, **FormEvent** o **MouseEvent**, que reciben como argumento genérico el tipo del elemento HTML:

```
import { useState, type ChangeEvent, type FormEvent } from 'react';
```

```
export function FormularioNombre() {
  const [nombre, setNombre] = useState('');

  function gestionarCambio(evento: ChangeEvent<HTMLInputElement>) {
    setNombre(evento.target.value);
  }

  function gestionarEnvio(evento: FormEvent<HTMLFormElement>) {
    evento.preventDefault();
    console.log(`Hola, ${nombre}`);
  }

  return (
    <form onSubmit={gestionarEnvio}>
      <input value={nombre} onChange={gestionarCambio} />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

Truco

Si no sabes qué tipo tiene un evento, escribe primero el manejador en línea, por ejemplo `onChange={(e) => {}}`, y pasa el cursor por encima de **e**. Visual Studio Code te mostrará el tipo exacto que debes usar al extraerlo a una función.

Ampliar las props de un elemento HTML

A menudo queremos crear un componente que envuelva un elemento HTML (un botón, un campo de texto...) y que acepte todos sus atributos habituales. Para no tener que escribirlos uno por uno, React proporciona el tipo **ComponentProps**:

```
import type { ComponentProps } from 'react';

type PropsBoton = ComponentProps<'button'> & {
  variante?: 'principal' | 'secundario';
};

export function Boton({ variante = 'principal', ...resto }: PropsBoton) {
  return <button className={`boton-${variante}`} {...resto} />;
}
```

El componente **Boton** acepta la prop propia **variante** y, además, cualquier atributo de un botón HTML (**onClick**, **disabled**, **type**, etc.), con su tipo correcto:

```
<Boton variante="secundario" disabled onClick={() => console.log('clic')}>
  Guardar
</Boton>
```

Resumen

- TypeScript complementa a JavaScript con un sistema de tipos completo y, en este tema, hemos visto cómo detectar errores de forma temprana gracias a su comprobación de tipos.
- JavaScript tiene tipado dinámico: el tipo de una variable puede cambiar durante la ejecución. TypeScript añade comprobación de tipos estática antes de ejecutar el código.

- Los tipos de JavaScript, como **number** y **string**, se pueden utilizar en TypeScript, que además conoce con precisión objetos como **Date** y añade tipos propios como **any**, **unknown** y **never**.
- **unknown** admite cualquier valor, pero obliga a estrechar el tipo mediante comprobaciones o guardas de tipo antes de usarlo.
- Se pueden crear nuevos tipos con alias de tipo e interfaces. Los alias de tipo pueden describir objetos, funciones o tipos de unión. Hemos utilizado un alias de tipo, con un tipo de unión, para tipar de forma estricta las props del componente de alerta.
- El símbolo **?** hace que una propiedad de un objeto o un parámetro de una función sea opcional, y un tipo existente se puede extender con el símbolo **&**.
- **as const** y **satisfies** permiten obtener tipos más precisos y ofrecen una alternativa moderna a los **enum**.
- El compilador de TypeScript se puede invocar desde la línea de comandos, lo que permite integrarlo en un proceso de integración continua. Realiza la transpilación a JavaScript y la comprobación de tipos, y se configura con un archivo **tsconfig.json**. TypeScript 7, publicado en 2026, ofrece un compilador nativo mucho más rápido.
- En React, hemos tipado las props, el estado (con el argumento genérico de **useState**) y los eventos de un componente.
- Hasta ahora solo hemos utilizado un Hook de React: **useState**. En el próximo tema aprenderemos muchos otros.

Preguntas

Responde a las siguientes preguntas para reforzar lo que has aprendido en este tema:

1. ¿Qué tipo infiere TypeScript para la variable **mensaje** en el siguiente código?

```
let mensaje = 'Hola';
```

2. ¿Por qué el siguiente código no produce ningún error de tipo, aunque **edad** no existe en el objeto?

```
let datos: any = { nombre: 'Ana' };
console.log(datos.edad.toFixed(2));
```

3. ¿Qué falla en el siguiente código y cómo se puede corregir sin usar **any**?

```
function mostrarNombre(valor: unknown) {
  console.log(valor.nombre);
}
```

4. En el componente **Alert**, ¿qué ventaja tiene declarar la prop como **type?: 'information' | 'warning'** en lugar de **type?: string**?
5. ¿Qué tipo tiene la variable de estado **cargando** en cada una de estas líneas?

```
const [cargando, setCargando] = useState(true);
const [cargando2, setCargando2] = useState<boolean>();
```

6. En un proyecto creado con la plantilla **react-ts** de Vite, la siguiente importación produce un error. ¿Por qué?

```
import { useState, ReactNode } from 'react';
```

Respuestas

1. TypeScript infiere el tipo **string** a partir del valor asignado.
2. Porque **datos** es de tipo **any**, y TypeScript no realiza ninguna comprobación de tipos sobre las variables **any**. El error solo aparecerá en tiempo de ejecución.
3. No se puede acceder a propiedades de un valor **unknown** sin estrechar antes su tipo. Una posible corrección:

```
function mostrarNombre(valor: unknown) {  
  if (  
    typeof valor === 'object' &&  
    valor !== null &&  
    'nombre' in valor  
  ) {  
    console.log(valor.nombre);  
  }  
}
```

4. Con la unión, TypeScript solo acepta los dos valores que entiende el componente: detecta erratas como **'warnig'**, y el editor autocompleta los valores válidos. Con **string** se aceptaría cualquier cadena y el error pasaría desapercibido.
5. En la primera línea, **cargando** es de tipo **boolean**. En la segunda, **cargando2** es de tipo **boolean | undefined**, porque no se ha pasado valor inicial.
6. Porque la plantilla activa la opción **verbatimModuleSyntax**, que obliga a importar los tipos con la palabra clave **type**. La importación correcta es: `import { useState, type ReactNode } from 'react';`